Deploy a Third-Party PFX Certificate to Mobile Devices for App Use with Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use (Microsoft Intune) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 5

What this solves . . . 5

Why a static third-party PFX is NOT a SCEP/PKCS (CA-issued) scenario . . . 5

The two correct paths — decision guide . . . 5

App-use and the iOS keychain — the nuance that decides success . . . 6

Mapping the real symptom . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 8

2.1 Licensing . . . 8

2.2 iOS/iPadOS enrollment . . . 8

2.3 The third-party PFX file + password . . . 8

2.4 The trusted root/intermediate chain . . . 8

2.5 Intune Certificate Connector (REQUIRED for Path A; not needed for Path B) . . . 8

2.6 Microsoft Graph PowerShell (for Path A import + the reporting scripts) . . . 9

2.7 The target app and its keychain requirement . . . 9

2.8 Pilot and broad device groups . . . 9

Checklist . . . 9

Global Requirements — Intune Solutions (category-level) 10

G1. Microsoft Entra ID tenant . . . 10

G2. Microsoft Intune subscription . . . 10

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

3. Implementation Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 11

Step 1 — Create the device groups (shared) . . . 11
Step 2 — Deliver the trusted root/intermediate chain (shared — do this first) . . . 11
PATH A — PKCS imported certificate (Certificate Connector + Graph import) . . . 11
PATH B — Custom iOS/iPadOS configuration profile carrying a PKCS12 payload . . . 13
Step 5 — Apply on a pilot device and validate the app can use the certificate (shared) . . . 13
Step 6 — Pilot gate . . . 13
Step 7 — Expand to production . . . 14
5. Testing Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 14
Test 1 — Trusted chain installed . . . 14
Test 2 — Identity (leaf) present on the device . . . 14
Test 3 — Profile reports Succeeded . . . 14
Test 4 — Certificate Connector decrypted the import (Path A only) . . . 14
Test 5 — The app can use the certificate (the real success criterion) . . . 15
Test 6 — Intended purpose matches (Path A, non-S/MIME app cert) . . . 15
Test 7 — Fleet report . . . 15
Pass/fail gate . . . 15
6. Troubleshooting Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 15
The core scenario — "PFX object exists in Graph but won't deploy" . . . 15
A. Certificate Connector unhealthy (Path A) . . . 16
B. Profile shows Error / Pending . . . 16
C. Chain missing — leaf present but untrusted/unusable . . . 16
D. Intended-purpose mismatch (non-S/MIME app cert) . . . 17
E. App can't access the keychain (delivered but not usable) . . . 17
F. Diagnostics quick reference . . . 17
G. When to escalate . . . 17
7. Best Practices — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 17
8. Deliverables Summary — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use 18
Documentation . . . 18
PowerShell scripts . . . 19
What the buyer receives . . . 19
Outcome . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use (Microsoft Intune)

White-label · Intune Solutions

A third-party PFX certificate — one that acts like a software/license credential an app consumes, not a standard TLS/SSL web certificate — needs to land on iOS/iPadOS devices through Microsoft Intune so a specific app can use it. The usual trap is reaching for a SCEP or PKCS (CA-issued) template: the request never appears the way the team expects and the deployment stalls, even though the PFX object already exists in Intune/Graph. The reason is simple once you see it — a static, externally-issued PFX is a **different delivery path** from a CA-issued certificate.

This is a complete, **deploy-it-today** package that corrects the path and delivers the certificate the right way, written for an administrator with no prior certificate-deployment experience.

What you get

- A plain-language **Solution Overview** explaining why a static third-party PFX is **not** a

SCEP/PKCS scenario, with a clear **Path A vs Path B** decision guide and the iOS **keychain-access** nuance that decides whether the app can actually use the certificate.

- A **Requirements Guide** with verify / implement-if-missing for every prerequisite (including the

Intune Certificate Connector and the trusted chain).

- An **Implementation Guide** of precise, click-by-click Intune steps in the required order — import

via Graph, the trusted-root chain, the **PKCS imported certificate** profile, the **custom PKCS12** alternative, and app validation.

- Five **PowerShell scripts** for discovery, per-device validation, fleet reporting, safe remediation,

and a Graph PFX-import helper.

- **Testing** and **Troubleshooting** guides, including the "PFX exists in Graph but won't deploy"

diagnostic tree.

What this delivers

The third-party PFX **imported into Intune** (Path A, via the Certificate Connector + Graph) or packaged in a **custom PKCS12 configuration profile** (Path B), delivered to iOS devices alongside the **trusted root/intermediate chain**, with the decision criteria and keychain guidance that make the certificate **usable by the target app** — not just present on the device.

The deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

A third-party PFX correctly delivered to iOS devices through the right profile type, with the app able to consume it — instead of stalling on the wrong SCEP/PKCS template — proven on a pilot before you scale.

1. Solution Overview — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

What this solves

A third-party PFX certificate — one that behaves like a software/license credential an app consumes, not a standard TLS/SSL web certificate — needs to reach iOS/iPadOS devices through Microsoft Intune so a specific app can use it. The certificate already exists as a file (a `.pfx`/`.p12` with a private key, protected by a password) and was issued by an external party, not by your internal certification authority (CA).

The common trap: teams reach for a SCEP or PKCS (CA-issued) profile, assign it, and nothing arrives — even when the PFX object is already visible in Intune/Graph. The deployment stalls because a static, externally-issued PFX uses a *different delivery path* from a CA-issued certificate. This solution explains why, then gives you the correct, repeatable way to import the PFX and deliver it to devices so the app can consume it.

Why a static third-party PFX is NOT a SCEP/PKCS (CA-issued) scenario

This is the single most important point in the package, so it leads the overview.

• A SCEP profile and a PKCS certificate profile both *issue a brand-new certificate* from an internal

CA at deployment time. The device (SCEP) or the Intune Certificate Connector (PKCS) talks to your CA, the CA mints a fresh certificate against a template, and that new certificate lands on the device. There is no way to "feed" a pre-existing third-party `.pfx` into either of those flows — they only know how to request *new* certificates from *your* CA.

• A third-party PFX is the opposite: it is a *static, already-issued* credential with its private key

baked in. There is no CA request to make and no template to issue against. You are *delivering an existing file*, not *issuing a new certificate*.

• This is why the usual symptoms appear: nothing shows up "on the CA," the SCEP/PKCS template never

fires, and the request the team is looking for never exists — because no certificate is being requested. The PFX object can sit in Intune/Graph indefinitely and still never reach a device, because importing the object and *assigning a delivery profile* are two separate actions.

The fix is to stop using the CA-issued (SCEP/PKCS *template*) path and use one of the two correct delivery paths below.

The two correct paths — decision guide

There are two supported enterprise ways to put a static third-party PFX onto an iOS device.

Path A — PKCS imported certificate (import to Intune via Graph, then assign an imported-PFX profile)

1. An admin imports the `.pfx` into Intune. The import encrypts the PFX to the Intune Certificate

Connector's public key and uploads it via Microsoft Graph (the imported/user PFX certificate flow — the same pattern as the PFXImport helper / Add-IntuneUserPfxCertificate).

1. You assign a `Devices > Configuration > Templates > PKCS imported certificate` profile to the user

(or group). At delivery time the Intune Certificate Connector *decrypts* the imported PFX and the identity is installed on the device.

• The Intune Certificate Connector is **required** for this path — it is what decrypts the imported

PFX. No healthy connector, no delivery.

• Caveat to flag up front: the imported-PFX profile's "intended purpose" options are **S/MIME-oriented**

(signing / encryption). That matters for a *non-S/MIME* app certificate — if your app does not consume an S/MIME identity, the intended-purpose framing may not match the app's expectation, which is one reason Path B is often cleaner for a plain app identity.

Path B — Custom iOS/iPadOS configuration profile carrying a PKCS12 (.p12/.pfx) payload

1. You build a `.mobileconfig` containing a PKCS12 payload (the `.p12`/`.pfx` plus its password).

2. You deliver it via `Devices > Configuration > Create > iOS/iPadOS > Templates > Custom`, uploading

the `.mobileconfig`.

• Best when you control the certificate and its password and simply need a specific identity present

on the device for an app. For an iOS app scenario this is frequently the most direct path — no Certificate Connector and no Graph import required.

Choosing between A and B

Use Path A (PKCS imported certificate) when…	Use Path B (Custom PKCS12 profile) when…
You want Intune to manage the imported PFX object centrally and the Certificate Connector is already healthy	You control the `.pfx` + password and want the most direct delivery for an app identity
The identity is genuinely S/MIME (signing/encryption)	The identity is a plain app credential (not S/MIME)
You need per-user PFX objects tracked in Intune/Graph	You are fine packaging the identity in a `.mobileconfig` you produce

In both paths you also deploy a **Trusted certificate** profile to install the root/intermediate **chain**. A leaf certificate is useless on the device if the issuing chain is not trusted — so the chain is not optional.

App-use and the iOS keychain — the nuance that decides success

Delivering the certificate is necessary but not always sufficient for the app to *use* it.

• On iOS, a profile-delivered identity lands in the **system keychain**.

• Whether the **target app** can read that identity depends on the app's **keychain access group /

entitlements**, or on an Intune** App Configuration policy** that points the app at the identity.

• If the app expects the identity in *its own* keychain access group, a plain config-profile PKCS12 may not be directly accessible to it — the app vendor's required delivery method governs.

• **Therefore:** make the Intune *delivery* correct (this is what this solution guarantees) and apply the decision criteria above; **defer app-internal specifics** (which keychain group the app reads, whether it needs an App Configuration key) to the app vendor's documentation.

Mapping the real symptom

> *"The PFX object exists in Graph but won't deploy."*

The missing piece is almost always a **PKCS imported certificate profile assigned to the user** (Path A) — or switching to the **custom-profile** path (Path B) — *plus a healthy Certificate Connector*. It is **not** a SCEP/PKCS *template* problem, and it is **not** a CA permissions/mapping problem. Importing the object and assigning a delivery profile are two separate steps; the object alone never reaches a device.

Benefits

• **Right path the first time** — stop burning cycles on SCEP/PKCS templates that cannot deliver a static PFX.

• **Two proven options** — imported-PFX vs custom PKCS12, with a clear decision guide for the app case.

• **App-ready** — the keychain-access guidance tells you what determines whether the app can use the identity, so you do not "deliver" a certificate the app can never read.

• **Validated** — per-device presence checks and a fleet report so you know the certificate actually landed.

Limitations (be honest with buyers)

• Path A **requires** the Intune Certificate Connector; if you cannot run a connector, use Path B.

• Whether the app can *consume* the identity is ultimately governed by the **app vendor** (keychain access group / App Configuration). This solution makes Intune delivery correct; it does not modify third-party apps.

• The imported-PFX profile's intended-purpose options are S/MIME-oriented — note this for a non-S/MIME app certificate.

• This is an **iOS/iPadOS** app-certificate scenario; other platforms use different payloads.

The standard deployment model

> ***Confirm/import the PFX → stand up a healthy Certificate Connector (Path A) → deliver the trusted chain → assign the imported-PFX profile (or the custom PKCS12 profile) → validate the app can use it → pilot → expand.*** *Pattern:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

> *Naming convention used throughout this package:* `<Purpose> – <Scope> – <Stage>`*. The objects you create are named exactly:* `PFX Imported – iOS App Cert – Pilot` *(Path A profile),* `Custom PKCS12 – iOS App Cert – Pilot` *(Path B profile),* `Trusted Root – Org CA – iOS` *(chain profile), and device groups* `iOS App Cert – Pilot` *and* `iOS App Cert – Broad`*. Reuse these verbatim in every step. Wherever a service identity is referenced, use the neutral placeholder* `Intune.Cert.<service>@<your-domain>` *— never a real tenant UPN.*

2.1 Licensing

Description: Intune (Plan 1) for the configuration profiles and (for Path A) the Certificate Connector. **Why:** Intune delivers the imported-PFX / custom / trusted-certificate profiles to devices. **Verify:** M365 admin center → **Billing** → **Licenses**. **Implement:** Assign M365 E3/E5 or Business Premium (Intune included).

2.2 iOS/iPadOS enrollment

Description: The target iOS/iPadOS devices are enrolled in Intune. **Why:** Configuration profiles apply only to enrolled devices. **Verify:** Intune → **Devices** → **All devices** (filter to iOS/iPadOS); run `PFXMOB-Discovery.ps1`. **Implement:** Enroll devices (Apple ADE, Apple Configurator, or user enrollment) per your model.

2.3 The third-party PFX file + password

Description: The actual `.pfx`/`.p12` file (private key included) and its password, from the issuing third party. **Why:** Both paths deliver *this exact file*; there is nothing to "issue" — you are delivering an existing credential. **Verify:** You can open the PFX with the password (e.g. import to a throwaway local store on an admin workstation and confirm it loads with its private key). **Implement:** Obtain the `.pfx` and password from the third party; store the password as a secret (never hardcode it in scripts or docs).

2.4 The trusted root/intermediate chain

Description: The root (and any intermediate) CA certificate(s) that issued the leaf in the PFX, as `.cer`/`.crt` files. **Why:** A leaf identity is unusable on the device unless its issuing chain is trusted; the chain is delivered by a separate **Trusted certificate** profile (`Trusted Root – Org CA – iOS`). **Verify:** Inspect the PFX issuer chain (e.g. on an admin workstation) and confirm you have each CA certificate above the leaf. **Implement:** Obtain the root/intermediate `.cer` files from the issuing party.

2.5 Intune Certificate Connector (REQUIRED for Path A; not needed for Path B)

Description: A healthy Intune Certificate Connector installed on a Windows server. **Why:** Path A (PKCS imported certificate) encrypts the PFX to the connector's public key on import, and the connector **decrypts** it at delivery time. No healthy connector = imported PFX never deploys. **Verify:** Intune → **Tenant administration** → **Connectors and tokens** → **Certificate connectors**; the connector shows **Active/Healthy** with a recent check-in. Run `PFXMOB-Discovery.ps1`. **Implement:** Install the Intune Certificate Connector on a domain-joined Windows server (`Tenant administration → Connectors and tokens → Certificate connectors → Add`), sign in, and confirm it reports healthy. (Skip this requirement entirely if you choose Path B.)

2.6 Microsoft Graph PowerShell (for Path A import + the reporting scripts)

Description: The Microsoft Graph PowerShell SDK on the admin workstation. **Why:** `PFXMOB-Import-Pfx.ps1` imports the PFX via Graph; the discovery/validation/report scripts read Intune state via Graph. **Verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a version. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Consent to the required scopes (`DeviceManagementConfiguration.Read.All`, and `DeviceManagementConfiguration.ReadWrite.All` for the import/remediation scripts) on first connect.

2.7 The target app and its keychain requirement

Description: The iOS app that must consume the certificate, and its vendor-stated method for accessing an identity (keychain access group / App Configuration key, or a custom profile it expects). **Why:** Delivering the identity to the system keychain does not guarantee the app can read it; the app vendor's required delivery method governs (see Overview, "App-use and the iOS keychain"). **Verify:** Check the app vendor's documentation for how the app expects the certificate to be provided. **Implement:** Follow the vendor's stated method; use the decision guide (Path A vs Path B) and, if required, add an Intune **App Configuration policy** for the app. App-internal specifics are deferred to the vendor.

2.8 Pilot and broad device groups

Description: Entra security groups `iOS App Cert – Pilot` (a few representative devices) and `iOS App Cert – Broad` (the rest). **Why:** Every deployment is piloted before fleet rollout. **Verify:** Entra → **Groups**; both groups exist. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

Checklist

- Intune licensing assigned
- iOS/iPadOS devices enrolled — `PFXMOB-Discovery.ps1`
- Third-party `.pfx`/`.p12` file + password in hand (password stored as a secret)
- Root/intermediate chain `.cer` files in hand
- Certificate Connector healthy (Path A only) — `PFXMOB-Discovery.ps1`
- Microsoft Graph PowerShell installed + scopes consented
- App vendor's keychain/App Configuration requirement understood

- `iOS App Cert – Pilot` and `iOS App Cert – Broad` groups created

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`). This is the resolution: it stops the SCEP/PKCS-template dead end and delivers the static third-party PFX the correct way.

> ***Model:*** *Confirm/import the PFX → deliver the trusted chain → assign the imported-PFX profile (Path A)* ***or*** *the custom PKCS12 profile (Path B) → validate the app can use it → pilot → expand. Choose Path A *or* Path B per the decision guide in the Overview; do both the shared steps (1, 2, 5) plus the path you chose.*
>
> ***Do NOT use a SCEP profile or a PKCS *certificate* (CA-issued) profile here.*** *Those issue new certificates from your CA and cannot deliver a pre-existing third-party PFX. If a SCEP/PKCS template is already assigned for this purpose, unassign it.*

Step 1 — Create the device groups (shared)

1. Entra → **Groups** → **New group** → **Security** → name `iOS App Cert – Pilot` (a few representative

iOS devices). **Create**.

1. Create `iOS App Cert – Broad` (the rest of the iOS fleet). **Create**.

Step 2 — Deliver the trusted root/intermediate chain (shared — do this first)

The leaf identity is useless on the device without a trusted chain, so deploy the chain before the identity.

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform:** iOS/iPadOS. **Profile type: Templates** → **Trusted certificate**. **Create**.
3. **Name:** `Trusted Root – Org CA – iOS`. **Next**.
4. **Configuration settings:** upload the **root** CA `.cer`. (If there is an intermediate, create a

second `Trusted certificate` profile for it, named `Trusted Intermediate – Org CA – iOS`.) **Next**.

1. **Assignments** → **Add groups** → `iOS App Cert – Pilot`. **Next** → **Create**.

PATH A — PKCS imported certificate (Certificate Connector + Graph import)

Use this when you want Intune to manage the imported PFX object and the Certificate Connector is healthy. See the decision guide in the Overview.

Step A1 — Confirm the Certificate Connector is healthy

1. Intune → **Tenant administration** → **Connectors and tokens** → **Certificate connectors**.

2. Confirm the connector shows **Active/Healthy** with a recent check-in. If not, repair it before continuing (see Troubleshooting A) — an unhealthy connector cannot decrypt the imported PFX, so the profile will never deliver.

Step A2 — Import the third-party PFX into Intune via Microsoft Graph

The PFX is imported (encrypted to the connector's public key) and uploaded as an imported/user PFX certificate object. Use the helper script:

1. On the admin workstation, run `PFXMOB-Import-Pfx.ps1`, supplying the `.pfx` path, its password (secure input — never hardcoded), the target user UPN placeholder (`Intune.Cert.<service>@<your-domain>`), and the intended purpose.

1. The script connects to Graph, retrieves the Certificate Connector's public key, encrypts the PFX, sets the certificate **thumbprint** and **intended purpose**, and uploads the object.

> ***Intended-purpose note:*** *the imported-PFX intended-purpose options are* ***S/MIME-oriented*** *(signing/encryption). For a non-S/MIME app certificate, set the value the app expects and be aware this framing may not match a plain app identity — if it doesn't fit, use Path B instead.*

1. Confirm the object exists: run `PFXMOB-Discovery.ps1` (it lists imported PFX objects) — this is the "PFX object exists in Graph" state. **The object alone does NOT reach a device; you must assign a profile (next step).**

Step A3 — Create and assign the PKCS imported certificate profile

This is the missing piece in the stalled scenario.

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.

2. **Platform:** iOS/iPadOS. **Profile type: Templates** → **PKCS imported certificate**. **Create**.

3. **Name:** `PFX Imported – iOS App Cert – Pilot`. **Next**.

4. **Configuration settings:**

• **Intended purpose:** match what you set on import (S/MIME signing / S/MIME encryption — see the note above for a non-S/MIME app cert).

• Leave the remaining settings at the documented defaults unless the app vendor specifies otherwise. **Next.**

1. **Assignments** → **Add groups** → `iOS App Cert – Pilot`. **Next** → **Create**.

> *The Certificate Connector decrypts the imported PFX and the identity is installed on assigned devices at their next sync. Proceed to Step 5 (validate the app).*

PATH B — Custom iOS/iPadOS configuration profile carrying a PKCS12 payload

Use this when you control the `.pfx` + password and need the most direct delivery for an app identity (often the cleanest for a non-S/MIME app cert). No Certificate Connector or Graph import required.

Step B1 — Build the .mobileconfig with a PKCS12 payload

1. Create a `.mobileconfig` containing a **PKCS12** payload that embeds the `.p12`/`.pfx` and its

password. (Apple Configurator or a known-good `.mobileconfig` template produces this; keep the password handling secure.)

1. Validate the `.mobileconfig` opens/parses correctly before uploading.

Step B2 — Create and assign the Custom profile

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform:** iOS/iPadOS. **Profile type: Templates** → **Custom**. **Create**.
3. **Name:** `Custom PKCS12 – iOS App Cert – Pilot`. **Next**.
4. **Configuration settings: Custom configuration profile name** = `iOS App Cert PKCS12`; upload

the `.mobileconfig` from Step B1. **Next**.

1. **Assignments** → **Add groups** → `iOS App Cert – Pilot`. **Next** → **Create**.

Step 5 — Apply on a pilot device and validate the app can use the certificate (shared)

1. On a pilot iOS device, force a sync: **Settings** → **General** → **VPN & Device Management** → (management

profile) → **Sync**, or push a sync from Intune (**Devices** → **(device)** → **Sync**).

1. Confirm the trusted chain and the identity are present: **Settings → General → VPN & Device

Management →** open the installed profile(s) and confirm the certificate(s) appear. Or run `PFXMOB-Validation.ps1` for the device.

1. **Open the target app and confirm it can use the certificate.** If the app cannot see the identity:

• Re-check the app vendor's required delivery method (keychain access group / App Configuration).

• If the vendor requires the identity in the app's own keychain access group, a plain custom PKCS12

may not be directly accessible — follow the vendor's method, or add an Intune **App Configuration policy** for the app (**Apps** → **App configuration policies** → **Add** → **Managed devices** → select the app → supply the vendor's keys).

• App-internal specifics are deferred to the app vendor; the Intune delivery above is correct.

Step 6 — Pilot gate

Do not expand until: the chain and identity are present on pilot devices (`PFXMOB-Validation.ps1`), the profile shows **Succeeded**, and the **app successfully uses the certificate**. Complete the **Testing Guide**. **Rollback:** if the pilot fails, unassign the profile(s) from `iOS App Cert – Pilot` (and remove the imported PFX object via Graph if Path A) — devices remove the certificate at next sync; no fleet impact.

Step 7 — Expand to production

1. Add `iOS App Cert – Broad` to the **Trusted certificate** profile and to your chosen delivery profile (`PFX Imported – iOS App Cert – Pilot` or `Custom PKCS12 – iOS App Cert – Pilot`); rename to `– Broad` if you prefer a separate broad profile.
1. Re-validate with `PFXMOB-Validation.ps1` and run `PFXMOB-Report.ps1` for fleet status.

Order of operations

`Groups → Trusted chain → (Path A: Connector healthy → Graph import → imported-PFX profile) OR (Path B: build .mobileconfig → custom profile) → Apply → Validate app → Pilot gate → Expand`

5. Testing Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

Run after the pilot profile applies (Implementation Step 5) and again before expanding.

Test 1 — Trusted chain installed

Do: On a pilot device, **Settings** → **General** → **VPN & Device Management** → open `Trusted Root – Org CA – iOS`; or run `PFXMOB-Validation.ps1`. **Expected:** The root (and intermediate, if any) CA certificate is present and trusted.

Test 2 — Identity (leaf) present on the device

Do: On the pilot device, open the delivery profile (`PFX Imported – iOS App Cert – Pilot` or `Custom PKCS12 – iOS App Cert – Pilot`) and confirm the certificate appears; or run `PFXMOB-Validation.ps1`. **Expected:** The leaf identity is installed and chains to the trusted root.

Test 3 — Profile reports Succeeded

Do: Intune → **Devices** → **Configuration** → open the delivery profile → **Device status**. **Expected:** Pilot devices show **Succeeded** (not Error/Pending).

Test 4 — Certificate Connector decrypted the import (Path A only)

Do: Intune → **Tenant administration** → **Connectors and tokens** → **Certificate connectors**; confirm the connector is healthy and the imported PFX deployed. **Expected:** Connector **Active/Healthy**; the imported-PFX profile delivers without connector errors.

Test 5 — The app can use the certificate (the real success criterion)

Do: Open the target app on the pilot device and exercise the function that consumes the certificate. **Expected:** The app reads/uses the identity successfully. If not, re-check the app vendor's keychain access / App Configuration requirement (Implementation Step 5) — delivery can be correct while the app still needs its vendor-specified access method.

Test 6 — Intended purpose matches (Path A, non-S/MIME app cert)

Do: Confirm the imported PFX intended purpose set at import matches the imported-PFX profile and what the app expects. **Expected:** Values match. A mismatch here is a common reason a non-S/MIME app cert deploys but is not consumed as expected — consider Path B if the S/MIME framing does not fit.

Test 7 — Fleet report

Do: Run `PFXMOB-Report.ps1`. **Expected:** A clear per-device certificate-deployment status across the targeted groups.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–3 and **Test 5** pass on `iOS App Cert – Pilot` (plus Test 4/6 for Path A)
Broad pass	Tests 1–3, 5, 7 pass after assigning `iOS App Cert – Broad`

6. Troubleshooting Guide — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

Use `PFXMOB-Discovery.ps1` (imported objects + connector + profiles), `PFXMOB-Validation.ps1` (per-device chain + identity), `PFXMOB-Report.ps1` (fleet status), and `PFXMOB-Remediation.ps1` (safe `-WhatIf` fixes).

The core scenario — "PFX object exists in Graph but won't deploy"

Work this tree top to bottom; the cause is almost always step 2 or 3.

1. **Are you using a SCEP or PKCS (CA-issued) *template*?**

• If yes → **that is the problem.** A SCEP/PKCS template issues a *new* certificate from your CA;

it cannot deliver a pre-existing third-party PFX. Unassign it and switch to a **PKCS imported certificate** profile (Path A) or a **custom PKCS12** profile (Path B). Do not keep checking CA mappings, CA permissions, local cert stores, or "the request on the CA" — no certificate is being requested, so none of those will ever show activity.

1. **Is a *delivery profile* assigned to the user/group?**

• The imported PFX object existing in Graph is **not** delivery. Confirm a `PFX Imported – iOS App Cert – Pilot` (Path A) or `Custom PKCS12 – iOS App Cert – Pilot` (Path B) profile is created **and assigned** to the device's group. Importing and assigning are separate steps — this missing assignment is the most common cause.

1. **Is the Certificate Connector healthy? (Path A)**

• The connector **decrypts** the imported PFX at delivery. If it is unhealthy, the imported-PFX profile silently never deploys. See section A.

1. **Is the trusted chain present?**

• If the leaf deploys but is untrusted/unusable, the root/intermediate chain is missing — deploy `Trusted Root – Org CA – iOS`. See section C.

1. **Does intended purpose match? (Path A, non-S/MIME app cert)**

• The imported-PFX intended-purpose options are S/MIME-oriented; a mismatch can deploy a cert the app does not consume as expected. See section D.

1. **Is the device synced and enrolled?**

• Force a sync; confirm the device is enrolled and in the assigned group. Run `PFXMOB-Remediation.ps1` (re-assign / re-trigger sync).

A. Certificate Connector unhealthy (Path A)

Symptom	Cause	Remediation
Imported-PFX profile never delivers	Connector offline / stale check-in	Intune → **Tenant administration** → **Connectors and tokens** → **Certificate connectors**; restart the connector service on the host; confirm it returns **Active/Healthy**; re-sync devices
Import succeeds but decrypt fails	Connector public key mismatch / wrong connector	Re-run `PFXMOB-Import-Pfx.ps1` so the PFX is encrypted to the *current* connector public key

B. Profile shows Error / Pending

• Confirm the device is enrolled and in `iOS App Cert – Pilot/– Broad`; force a sync.

• Re-check the profile assignment and platform (iOS/iPadOS). Run `PFXMOB-Remediation.ps1`.

C. Chain missing — leaf present but untrusted/unusable

• Deploy/confirm `Trusted Root – Org CA – iOS` (and an intermediate profile if the chain has one).

• Verify with `PFXMOB-Validation.ps1` (checks chain + identity). A leaf without its chain is the classic "cert is there but nothing works" symptom.

D. Intended-purpose mismatch (non-S/MIME app cert)

• The imported-PFX intended purpose is S/MIME-oriented (signing/encryption). If the app expects a plain app identity, the framing may not fit — set what the app expects, or use **Path B** (custom PKCS12), which avoids the imported-PFX intended-purpose model entirely.

E. App can't access the keychain (delivered but not usable)

• The identity is in the **system keychain**, but the app may require it in **its own keychain access group **or via an** App Configuration policy**.

• Re-check the app vendor's required delivery method. If the vendor needs the app's keychain access group, a plain custom PKCS12 may not be directly accessible — follow the vendor's method or add an Intune **App Configuration policy** for the app (**Apps** → **App configuration policies** → **Add** → **Managed devices**).

• App-internal specifics are deferred to the app vendor; verify Intune delivery is correct first (`PFXMOB-Validation.ps1`), then hand off to the vendor.

F. Diagnostics quick reference

```
# Imported PFX objects + connector + profiles (read-only)
.\PFXMOB-Discovery.ps1
# Per-device chain + identity + profile state (read-only)
.\PFXMOB-Validation.ps1 -DeviceName 'iPad-Pilot-01'
# Fleet certificate-deployment status (read-only)
.\PFXMOB-Report.ps1
```

In Intune, check `Tenant administration > Connectors and tokens > Certificate connectors` (health) and `Devices > Configuration > (profile) > Device status` (per-device result).

G. When to escalate

If the correct delivery profile is assigned, the Certificate Connector is healthy (Path A), the chain is present, the device is synced, and the certificate is confirmed installed by `PFXMOB-Validation.ps1` but the **app still cannot use it**, the remaining cause is app-internal (keychain access group / App Configuration). Collect the validation output and engage the **app vendor** with the vendor's required certificate-delivery method.

7. Best Practices — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

• **Never use SCEP/PKCS *templates* for a static third-party PFX.** Those issue *new* certificates

from your CA; they cannot deliver a pre-existing file. Use a PKCS imported certificate profile (Path A) or a custom PKCS12 profile (Path B). This is the single most important practice.

• **Deliver the chain before the leaf.** A leaf identity is useless without a trusted root/intermediate chain — assign `Trusted Root – Org CA – iOS` first.

• **Remember the object-vs-profile distinction.** An imported PFX object existing in Graph does *not* reach a device. A delivery *profile* must be assigned to the user/group; importing and assigning are two separate actions.

• **Keep the Certificate Connector healthy (Path A).** It decrypts the imported PFX at delivery time; if it is unhealthy, the imported-PFX profile silently never deploys.

• **Match intended purpose to the app.** The imported-PFX intended-purpose options are S/MIME-oriented; for a non-S/MIME app cert set what the app expects, and switch to Path B if the S/MIME framing does not fit.

• **Confirm the app can actually use the cert.** "Delivered" is not "usable" — verify the app reads the identity, and apply the vendor's keychain access group / App Configuration requirement.

• **Defer app internals to the vendor.** Make Intune delivery correct; let the app vendor own which keychain group the app reads.

• **Protect the PFX password.** Treat it as a secret — secure input in scripts, never hardcoded, never in docs or logs.

• **Pilot before you expand.** Validate on `iOS App Cert – Pilot` (chain + identity + app use) before adding `iOS App Cert – Broad`; keep a clean rollback by unassigning the profile.

• **Use the naming convention.** `<Purpose> – <Scope> – <Stage>` across profiles and groups keeps the deployment auditable and repeatable.

8. Deliverables Summary — Deploy a Third-Party PFX Certificate to Mobile Devices for App Use

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Why a static third-party PFX is NOT SCEP/PKCS; Path A vs Path B decision guide; app-keychain nuance
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered Intune steps — chain, import, imported-PFX profile, custom PKCS12 profile, app validation
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	"PFX exists in Graph but won't deploy" diagnostic tree, connector health, intended-purpose mismatch, missing chain, app can't access keychain
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`PFXMOB-Discovery.ps1`	Inventories imported PFX objects, Certificate Connector status, and existing certificate profiles (read-only)
`PFXMOB-Validation.ps1`	Per-device: confirms the trusted chain + identity are present and the profile applied (read-only)
`PFXMOB-Report.ps1`	Fleet-wide certificate-deployment status (read-only)
`PFXMOB-Remediation.ps1`	Safe remediation (`-WhatIf` default): re-assign/repair the profile and re-trigger device sync
`PFXMOB-Import-Pfx.ps1`	Helper that imports a third-party PFX to Intune via Microsoft Graph (encrypts to the Certificate Connector public key; sets thumbprint + intended purpose; secure password input)

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

A third-party PFX correctly imported into Intune and delivered to iOS devices through the right profile type — with the app able to consume it — instead of stalling on the wrong SCEP/PKCS template, proven on a pilot before scaling.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PFXMOB-Discovery.ps1

Purpose: Establishes the "what exists today" baseline for delivering a third-party PFX to iOS: 1. Lists imported/user PFX certificate objects already in Intune/Graph (the "PFX exists in Graph" state) so you can confirm the import without assuming it delivered. 2. Reports Intune Certificate Connector health (REQUIRED for the PKCS imported-certificate path — it decrypts the imported PFX at delivery time). 3. Lists existing iOS certificate-related configuration profiles (PKCS imported certificate, trusted certificate, and custom profiles) so you can see what is already assigned. HOW TO RUN Install the Graph SDK once: Install-Module Microsoft.Graph -Scope CurrentUser Then: powershell -ExecutionPolicy Bypass -File .\PFXMOB-Discovery.ps1 Sign in when prompted (read scopes only). EXPECTED RESULTS A list of imported PFX objects, the Certificate Connector(s) and their state, and the iOS certificate/custom profiles present in the tenant. No changes are made. .NOTES Intune Solutions. Read-only. Uses documented Microsoft Graph deviceManagement endpoints via Invoke-MgGraphRequest. Reports against Microsoft Graph beta for the userPfxCertificates / ndesConnectors resources where v1.0 coverage is incomplete (clearly noted). > [CmdletBinding()] param()

How to run: Install the Graph SDK once: Install-Module Microsoft.Graph -Scope CurrentUser Then: powershell -ExecutionPolicy Bypass -File .\PFXMOB-Discovery.ps1 Sign in when prompted (read scopes only). EXPECTED RESULTS A list of imported PFX objects, the Certificate Connector(s) and their state, and the iOS certificate/custom profiles present in the tenant. No changes are made. .NOTES Intune Solutions. Read-only. Uses documented Microsoft Graph deviceManagement endpoints via Invoke-MgGraphRequest. Reports against Microsoft Graph beta for the userPfxCertificates / ndesConnectors resources where v1.0 coverage is incomplete (clearly noted). > [CmdletBinding()] param()

PFXMOB-Import-Pfx.ps1

Purpose: Path A helper. Uploads an EXISTING third-party .pfx/.p12 to Intune as a user PFX certificate object so a "PKCS imported certificate" profile can later deliver it to iOS devices. It does NOT issue a new certificate (that is SCEP/PKCS, the WRONG path for a static PFX).

How to run: Install once: Install-Module Microsoft.Graph -Scope CurrentUser Install-Module IntunePfxImport -Scope CurrentUser # Microsoft PFXImport helper Then (you will be prompted for the PFX password securely): powershell -ExecutionPolicy Bypass -File .\PFXMOB-Import-Pfx.ps1 `-PfxPath 'C:\secure\app-cert.pfx'` -UserUpn 'Intune.Cert.<service>@<your-domain>'` -IntendedPurpose smimeEncryption Or pass the password non-interactively as a SecureString via -PfxPassword (still never plain text). EXPECTED RESULTS The third-party PFX imported into Intune as a userPfxCertificate object for the target user. Confirm with PFXMOB-Discovery.ps1, then assign a "PKCS imported certificate" profile (PFX Imported - iOS App Cert - Pilot) to the user/group — the object alone does NOT deploy. .NOTES Intune Solutions. Requires write scope. Privacy: use the neutral placeholder Intune.Cert.<service>@<your-domain> — never a real tenant UPN. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [ValidateScript({ Test-Path $_ })] [string] $PfxPath, [Parameter(Mandatory)] [string] $UserUpn, S/MIME-oriented options per the imported-PFX model. Choose what the app expects. [ValidateSet('unassigned','smimeEncryption','smimeSigning')] [string] $IntendedPurpose = 'smimeEncryption', [System.Security.SecureString] $PfxPassword)

PFXMOB-Remediation.ps1

Purpose: Addresses the most common cause of "PFX exists in Graph but won't deploy": the delivery profile is not assigned to the device's group, or the device has not synced. This script: 1. Confirms the delivery profile exists and is assigned to the target group; if missing, ADDS the group assignment (Path A: PKCS imported certificate profile; Path B: custom PKCS12 profile). 2. Re-triggers a sync on the targeted managed devices so the profile re-applies. It is intentionally conservative: by default it only REPORTS what it would do. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\PFXMOB-Remediation.ps1 ` -ProfileName 'PFX Imported - iOS App Cert - Pilot' -GroupName 'iOS App Cert - Pilot' Apply changes: add -Apply EXPECTED RESULTS WhatIf: a list of the assignment/sync actions that WOULD be taken. Apply : the group assignment added (if missing) and a sync requested on member devices. .NOTES Intune Solutions. Write actions require -Apply and the DeviceManagementConfiguration.ReadWrite.All / DeviceManagementManagedDevices.PrivilegedOperations.All scopes. Documented Graph endpoints (deviceConfigurations/assign, managedDevices/syncDevice). > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string] $ProfileName, [Parameter(Mandatory)] [string] $GroupName, [switch] $Apply) -WhatIf semantics: default to preview unless -Apply is given. if (-not $Apply) { $WhatIfPreference = $true; Write-Host "[WhatIf default] No changes will be made. Pass -Apply to execute." -ForegroundColor Yellow }

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\PFXMOB-Remediation.ps1 ` -ProfileName 'PFX Imported - iOS App Cert - Pilot' -GroupName 'iOS App Cert - Pilot' Apply changes: add -Apply EXPECTED RESULTS WhatIf: a list of the assignment/sync actions that WOULD be taken. Apply : the group assignment added (if missing) and a sync requested on member devices. .NOTES Intune Solutions. Write actions require -Apply and the DeviceManagementConfiguration.ReadWrite.All / DeviceManagementManagedDevices.PrivilegedOperations.All scopes. Documented Graph endpoints (deviceConfigurations/assign, managedDevices/syncDevice). > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string] $ProfileName, [Parameter(Mandatory)] [string] $GroupName, [switch] $Apply) -WhatIf semantics: default to preview unless -Apply is given. if (-not $Apply) { $WhatIfPreference = $true; Write-Host "[WhatIf default] No changes will be made. Pass -Apply to execute." -ForegroundColor Yellow }

PFXMOB-Report.ps1

Purpose: Produces a per-device deployment summary across the certificate profiles used by this solution (the trusted chain profile and the delivery profile), so you can see at a glance which devices have the certificate delivered and which are pending/failed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PFXMOB-Report.ps1 Optional: -ProfileNames @('Trusted Root - Org CA - iOS','PFX Imported - iOS App Cert - Pilot') Optional: -CsvPath .\pfx-report.csv (also writes a CSV) EXPECTED RESULTS A table of device | profile | state, plus a per-state count. No changes are made. .NOTES Intune Solutions. Read-only. Documented Graph endpoints (deviceConfigurations + deviceStatuses). > [CmdletBinding()] param([string[]] $ProfileNames = @('Trusted Root - Org CA - iOS', 'PFX Imported - iOS App Cert - Pilot', 'PFX Imported - iOS App Cert - Broad', 'Custom PKCS12 - iOS App Cert - Pilot', 'Custom PKCS12 - iOS App Cert - Broad'), [string] $CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\PFXMOB-Report.ps1 Optional: -ProfileNames @('Trusted Root - Org CA - iOS','PFX Imported - iOS App Cert - Pilot') Optional: -CsvPath .\pfx-report.csv (also writes a CSV) EXPECTED RESULTS A table of device | profile | state, plus a per-state count. No changes are made. .NOTES Intune Solutions. Read-only. Documented Graph endpoints

(deviceConfigurations + deviceStatuses). > [CmdletBinding()] param([string[]] $ProfileNames = @('Trusted Root - Org CA - iOS', 'PFX Imported - iOS App Cert - Pilot', 'PFX Imported - iOS App Cert - Broad', 'Custom PKCS12 - iOS App Cert - Pilot', 'Custom PKCS12 - iOS App Cert - Broad'), [string] $CsvPath)

PFXMOB-Validation.ps1

Purpose: For a target iOS device, confirms (via Microsoft Graph) that: 1. The device exists and is enrolled/managed. 2. The trusted certificate profile (Trusted Root - Org CA - iOS) reports Succeeded. 3. The delivery profile (PFX Imported - iOS App Cert - Pilot, OR Custom PKCS12 - iOS App Cert - Pilot) reports Succeeded for the device. This validates the Intune DELIVERY only. Whether the target app can READ the identity from the keychain is governed by the app vendor (keychain access group / App Configuration) and is confirmed by opening the app — see the Troubleshooting Guide, section E. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PFXMOB-Validation.ps1 -DeviceName 'iPad-Pilot-01' (Or pass -DeviceId for an exact Intune managed-device id.) EXPECTED RESULTS Per-profile deployment status for the device; a clear PASS when both the chain profile and the delivery profile show Succeeded. .NOTES Intune Solutions. Read-only. Documented Graph endpoints (deviceManagement/managedDevices and per-device configuration state). > [CmdletBinding(DefaultParameterSetName='ByName')] param([Parameter(ParameterSetName='ByName', Mandatory)] [string] $DeviceName, [Parameter(ParameterSetName='ById', Mandatory)] [string] $DeviceId, [string] $ChainProfileName = 'Trusted Root - Org CA - iOS', [string[]] $DeliveryProfileNames = @('PFX Imported - iOS App Cert - Pilot','Custom PKCS12 - iOS App Cert - Pilot'))

How to run: powershell -ExecutionPolicy Bypass -File .\PFXMOB-Validation.ps1 -DeviceName 'iPad-Pilot-01' (Or pass -DeviceId for an exact Intune managed-device id.) EXPECTED RESULTS Per-profile deployment status for the device; a clear PASS when both the chain profile and the delivery profile show Succeeded. .NOTES Intune Solutions. Read-only. Documented Graph endpoints (deviceManagement/managedDevices and per-device configuration state). > [CmdletBinding(DefaultParameterSetName='ByName')] param([Parameter(ParameterSetName='ByName', Mandatory)] [string] $DeviceName, [Parameter(ParameterSetName='ById', Mandatory)] [string] $DeviceId, [string] $ChainProfileName = 'Trusted Root - Org CA - iOS', [string[]] $DeliveryProfileNames = @('PFX Imported - iOS App Cert - Pilot','Custom PKCS12 - iOS App Cert - Pilot'))